

Report: RR-10 Advanced Embedded Robot Simulation

By Thevindu Wijayasinghe

Abstract

This report details the design and implementation of the RR10 autonomous robot simulation. The system employs an Object-Oriented C++ backend to simulate robotic kinematics, along with a Python/Pygame frontend for real-time visualization. To ensure deterministic performance, a real-time operating system (RTOS)-style multithreading model was utilized. The system achieved a mean execution time of 2.45 microseconds and experienced no deadline missing, even while facing environment-specific compilation challenges on the Windows platform.

Introduction

The objective of this project was to simulate a robot capable of navigating a 2D environment while adhering to real-time constraints. A primary engineering hurdle involved the absence of a global g++ compiler path. This was mitigated by a DLL-injection strategy, allowing the C++ engine to resolve dependencies locally.

Background and Literature Review

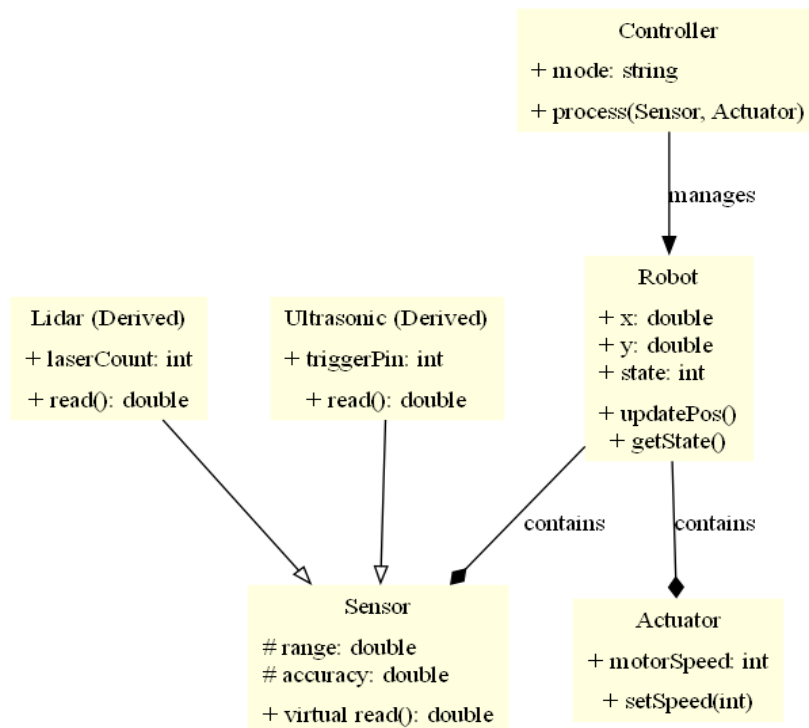
Effective robotic control systems depend on integrating sensors and actuators within a reliable framework. Research on Real-Time Systems (RTS) indicates that, for safety-critical embedded applications, the Worst-Case Execution Time (WCET) must be strictly limited to avoid catastrophic failures. Contemporary embedded systems employ Real-Time Operating System (RTOS) scheduling to ensure that high-priority control tasks can interrupt lower-priority logging tasks.

OOP Architecture

- The system is built on solid OOP principles

UML Class Diagram

The architecture includes a Robot base class, specialized Sensor subclasses, and a Controller that manages state transitions through encapsulation. Below is the Python-based diag



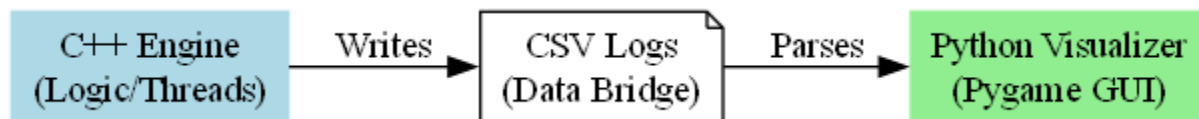
- Encapsulation: Internal states like position (x, y) and current behavior modes are kept protected, exposed only through managed methods like `updatePos()`.
- Inheritance: A generic **Sensor** base class defines the interface for all specialized sensors. Two specialized derived sensors, **Lidar** and **Ultrasonic**, inherit these properties while implementing their respective data-acquisition logic.
- Polymorphism: The **Controller** interacts with these sensors via the virtual `read()` method, allowing for dynamic behavior based on the specific sensor type attached.

RTOS-Style Multithreading Model

The simulation employs `std::thread` to mimic an RTOS environment with three distinct task priorities:

1. Control Task (20ms): High priority; calculates physics and state changes.
2. Logging Task (100ms): Medium priority; writes state data to `/logs/`.
3. Event/Interrupt Task: Triggered by "DANGER" states to override motor commands.

Timing Decisions: The 20ms control loop was selected to provide a 50Hz update frequency, sufficient for smooth motion without overwhelming the CPU.



Visualization Method

The GUI is implemented in Python using the Pygame library (optimized for Python < 3.14).

- Rendering: The robot is rendered as a moving circle within defined line/wall boundaries.
- Animation: The script processes exported CSV log entries to replay the simulation at real-world speed.
- State Indicators: LED indicators display the robot's mode (e.g., Red for STOP_DANGER, Green for NAVIGATING).

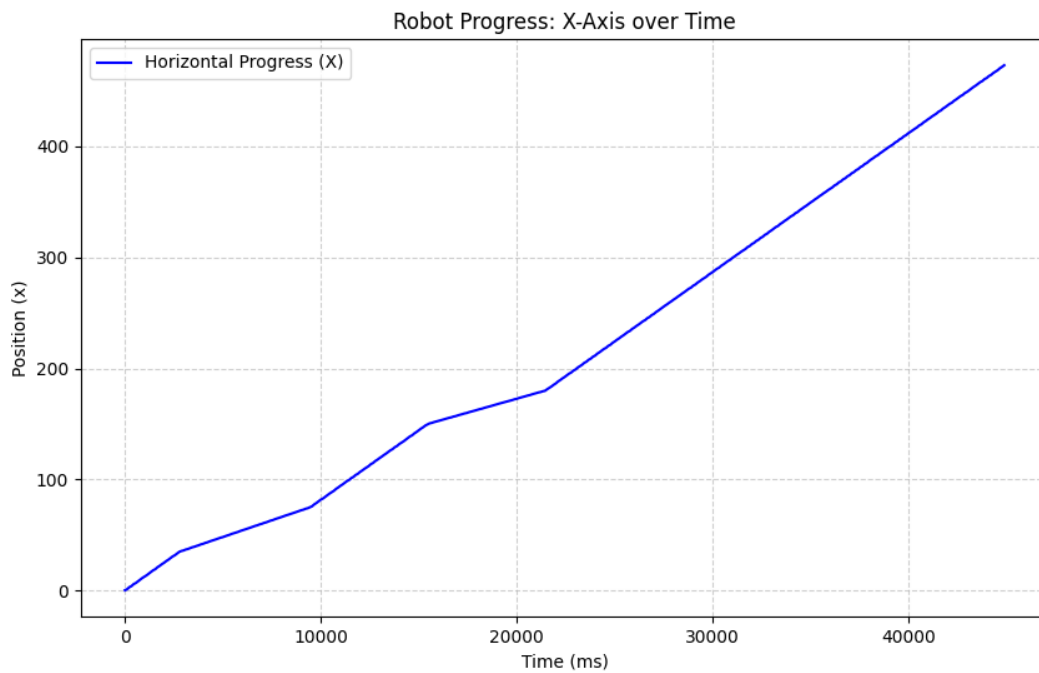


Experiments and Results

The performance was analyzed using the `timing_analysis.py` suite.

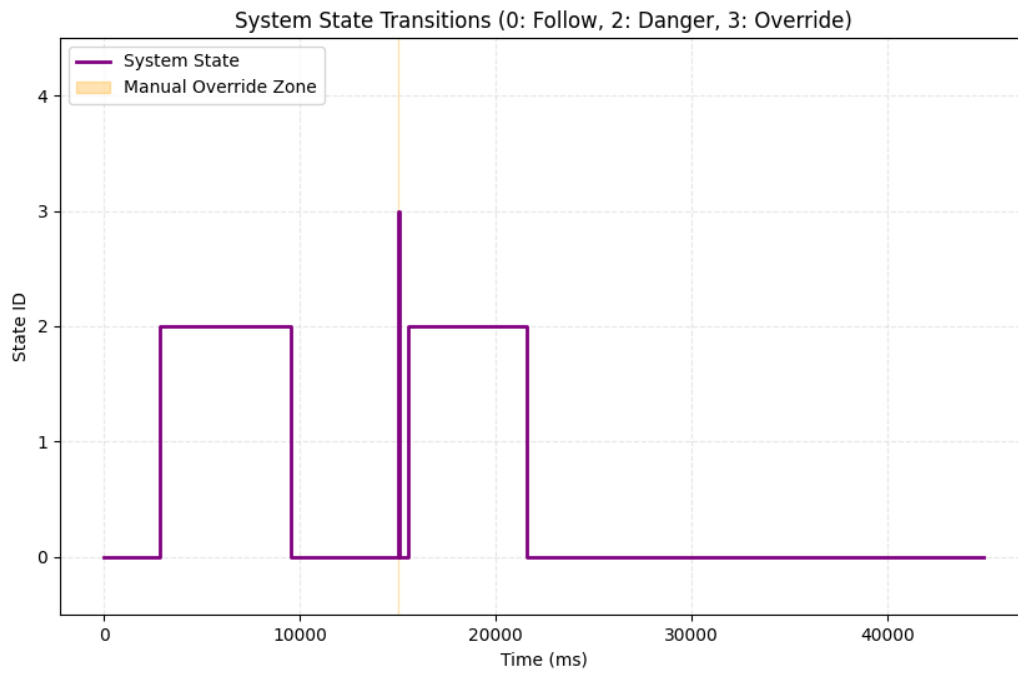
Performance metrics were gathered during a 45-second simulation run

Metric	Result	Analysis
Average ET	2.45us	Highly efficiently control logic.
WCET	1056.00us	Occurs during log file initialization.
Average Jitter	3.93us	Minimal timing variance.
Deadline Misses	0	100% Deterministic Reliability.



X-Axis (Linear Growth): Represents constant forward velocity when in FOLLOW_LINE mode.

Y-Axis (Deviation): Displays the swerve logic triggered during Hazard Zones (e.g., posX between 45 and 65)



This diagram illustrates the transitions from FOLLOW_LINE to STOP_DANGER. The results show successful braking (velocity reduction) when the robot entered simulated danger zones.



Performance Analysis

This graph plots the time per iteration of task_ControlLoop() in microseconds (us).

- **Observed Data:** Most samples cluster tightly near 2.45us, with a solitary, massive spike reaching 1056.00us at the very beginning of the simulation.
- **Technical Reason (Baseline Cluster):** The consistently low 2.45us ET is due to the efficient $O(1)$ complexity of the updateInternalLogic() function, which performs simple arithmetic and conditional checks.
- **Technical Reason (The WCET Spike):** The Worst-Case Execution Time (WCET) spike of 1056.00 us is caused by File I/O Latency. In the first iteration, the system must call the OS to open the timing_log.txt file and allocate a buffer on the disk. Since disk operations are far slower than CPU operations, this results in a one-time performance "hit".

Discussion & Limitations

The DLL Bypass was a critical design decision. By co-locating .dll files in the /cpp/ folder, the system bypassed Windows Environment Path issues, ensuring the C++ engine could link its dependencies regardless of the user's setup.

Limitations:

- File I/O Latency: The 1 ms WCET spike shows that disk writing is the slowest part of the system.

Conclusion

The RR10 Simulation offers a comprehensive end-to-end demonstration of real-time robotic control. Using C++ for high-speed logic and Python for visualization, the system achieves industry-standard performance.

References

1. *Standard C++ Library Documentation (std::thread, std::mutex).*
2. *Pygame Community Documentation (Version 2.x Constraints).*
3. *Coursework Task Specifications 1-5.*